

Revon: Git-Inspired Merkle Hash Trie Versioning for Structured Data

Atharva Sheersh Pandey¹, Adhyan Jain¹, Poornima Nedunchezian²
atharva.sheersh2024@vitstudent.ac.in; adhyan.jain2024@vitstudent.ac.in;
poornima.n@vit.ac.in
School of Computer Science and Engineering
VIT Vellore

Abstract: Versioning structured data means keeping exact historical states, handling sparse updates, and comparing versions without copying the full dataset each time. Revon is a Python prototype built on an immutable, fixed-depth Merkle hash trie. Its hybrid variant, Revon-H, stores addressed changesets and chooses between log aggregation and hash-pruned differencing. We tested Snapshot, Log-only, forced-Merkle Revon-M, Revon-H, and Dolt 2.3.1 on deterministic workloads of up to 100,000 rows, across 540 successful executions. For the tested workflows, median paired Dolt/Revon-H diff-latency ratios ranged from 7.14 to 625.10, and all seven-trial 95% bootstrap intervals favored Revon-H latency. Those are interface-level results: the comparison includes different APIs and does not isolate tree or index performance. A separate three-seed study could not resolve the 50-commit ablation result. We ran the experiments on one Windows host and under WSL2 on that same computer. The results show workload-dependent trade-offs for this prototype; they do not establish general or production-level superiority.

Index Terms: data versioning, Merkle hash trie, hybrid differencing, content-addressed storage, benchmark methodology

I. INTRODUCTION

Structured-data version control has to do more than archive files. It must preserve exact versions, apply sparse updates without rebuilding the whole state, restore earlier versions, and compare states efficiently. Git offers a useful starting point: immutable objects named by their content, linked through parent commits [1]. Its objects are organized around files and directories, however. Record stores have a different shape: keys may remain stable while update density changes, and two versions may differ by a few edits or by a long commit history.

Revon asks whether a fixed-depth Merkle hash trie can combine incremental structural sharing with an operation index. The main variant, Revon-H, uses the log path for short or sparse intervals and switches to Merkle comparison after a calibration threshold fixed before evaluation. SQLite supplies atomic, durable storage; it does not define Revon’s versioning algorithm.

We use Dolt as the production comparator because it combines SQL, a Git-style commit graph, and content-addressed Prolly Trees [2], [3]. Revon is a smaller research prototype, not a feature-for-feature replacement for Dolt. Our results compare Revon’s in-process Python API with a Dolt CLI workflow; they do not isolate the performance of either tree structure.

A. Research Questions

- RQ1: How do Revon-H’s latency and storage compare with Snapshot, Log-only, and Dolt across dataset sizes, mutation densities, and update localities?
- RQ2: Does choosing between log and Merkle diff improve latency over forced-Merkle Revon-M while preserving correctness?
- RQ3: How do branching factor and trie depth affect latency, storage, and the amount of work examined?

B. Contributions

- We implement a canonical SHA-256-addressed, fixed-depth trie with incremental rewriting of paths affected by multi-key updates.
- We design a hybrid diff policy that combines addressed changesets with hash-pruned tree comparison.
- We build an atomic SQLite repository for objects, commits, aliases, measurements, and HEAD, and verify object hashes after reopening it.
- We compare five systems on shared deterministic workloads, using common diff and checkout outputs. We report Dolt’s two initial-import workflows separately, randomize and balance execution order, sample process-tree RSS externally, retain raw evidence, and audit its integrity.
- We measure how branching factor and depth affect the fixed trie, rather than assuming that $b=8$ and $d=4$ are universally best.

II. RELATED WORK

A. Content Addressing and Persistent Structures

Merkle’s authenticated-tree construction showed that a compact root digest can commit to a larger collection [4]. Git and IPFS apply content addressing to software history and networked content [1], [6]. Persistent-data-structure theory formalized path copying and access to earlier states [5]. Later persistent dictionaries explore ordered, disk-oriented alternatives: stratified B-trees [16] and fully persistent B-trees [19]. Revon instead routes hashed keys through a fixed-depth trie, trading ordered range access for hash-pruned comparison. Scientific array systems such as SciDB and TimeArr preserve versions using array-specific storage layouts and deltas [17], [18]. TardisDB integrates version access with a main-memory database, while Saquib et al. study replicated versioned struc-

tures across wide-area systems [20], [21]. These systems illuminate adjacent design choices, but they do not compare Revon’s log and Merkle diff paths over the same history.

Merkle Search Trees combine search-tree ordering with Merkle authentication to reconcile state [7]. Revon also prunes comparisons by hash, but uses fixed-depth routing and a centralized linear history rather than a replicated CRDT. LBFS introduced content-defined chunking for redundancy detection [8]; Dolt and Noms use related boundary strategies. Ordered indexes preserve range operations, while content-defined boundaries can improve deduplication when data shifts. Revon’s current benchmark measures neither capability.

B. Structured Dataset Versioning

Dataset-versioning research covers collaboration, storage layout, and retrieval. DataHub studies collaborative dataset history at scale [9]. Decibel brings branching into a relational engine and compares version-first, tuple-first, and hybrid layouts [10]. OrpheusDB adds versioning to a conventional DBMS and optimizes version retrieval through partitioning [11]. Bhattacharjee et al. formalize the storage-versus-reconstruction trade-off [12], while Delta Lake uses transaction logs

and checkpoints for ACID tables [13]. More recent work studies how to choose materialized versions and deltas over a version graph, subject to storage and retrieval costs [26]. A FAIR-compliant proposal focuses on identifying and describing dataset releases consistently [27]; it complements storage research but does not provide a diff algorithm. Revon addresses a narrower question with a key/value prototype and does not solve these broader dataset-management problems.

C. System Design Gap

Noms stores structured values in a Merkle DAG and supports diff and synchronization [14]. ForkBase combines content addressing with fork and deduplication semantics [15]. Dolt’s content-addressed Prolly Trees combine ordered B-tree-like indexing, structural sharing, and native diff [2], [3]. Git4Data is another nearby direction: it presents SQL-level snapshot, branch, diff, and merge operations over a relational engine [29]. The available Git4Data manuscript is an arXiv preprint, so we do not treat its results here as peer-reviewed or independently reproduced evidence. Revon has a narrower scope: it is a single-writer, linear-history key/value prototype with fixed-depth hash routing, not a SQL database or branching system.

TABLE I. Related work and the scope of Revon.

System or study	Representation or focus	Relevance to Revon
Persistent DS [5]	Path copying	Formal structural-sharing basis
Git [1]	Object DAG	Identity and commit ancestry; file-oriented
IPFS [6]	Merkle DAG	General content-addressed versioning
MST [7]	Merkle search tree	Authenticated ordered reconciliation
DataHub [9]	Version graph	Collaborative dataset motivation
Decibel [10]	Relational layouts	Native database branching
OrpheusDB [11]	Partitioned versions	Storage/retrieval optimization
Delta Lake [13]	Log/checkpoints	ACID time travel at data-lake scale
Noms [14]	Merkle DAG	Structured content addressing
ForkBase [15]	Forkable CAS	Fork and deduplication semantics
DoltHub [2,3]	Prolly Trees	Mature production comparator

D. Design Implications and Research Positioning

Versioning systems face three recurring costs: retaining history, creating a version, and later reconstructing or comparing versions. Snapshots simplify retrieval by copying unchanged records. Logs defer work until replay or checkpointing. Path copying creates only changed paths and shares the rest [5], [12]. Decibel and OrpheusDB show that physical layout affects both version creation and retrieval [10], [11]; graph-based analyses make the same storage/retrieval trade-off explicit [26]. Reproducible-research work addresses a separate problem: preserving builds and artifacts for later inspection does not show that one storage algorithm is faster [24]. For that reason, Revon reports import, commit, checkout, diff, and storage measurements separately.

Different research areas also give “change” different meanings. Differential dataflow incrementally recomputes query results as timestamped inputs change [23]. Data-provenance research asks why and where output facts were derived [22], and DSLog stores and queries fine-grained array lineage [28]. Explain-Da-V uses data transformations to explain semantic

dataset changes [25]. Baseline uses operation-based differencing that can account for refactorings and schema changes [30]. Revon reports a more limited result: which keys were added, removed, or changed in value between two states. It does not infer causes, transformation lineage, query impact, or schema intent. These systems offer useful context, but their outputs and workloads cannot be compared directly with Revon’s key/value diff timings.

Revon’s contribution is a controlled comparison of two correct diff paths over the same durable history, selected using properties of the requested interval. Each Revon-H commit stores both a Merkle root and an addressed changeset, so the choice of diff path does not change version semantics. We hypothesize that log aggregation suits short, sparse intervals and that hash pruning may help after longer operation sequences. The measured crossover depends on the workload and configuration. Work such as Git4Data also shows that database-native version control remains an active area [29], which narrows our claim: Revon evaluates adaptive differencing for a fixed-depth hash trie; it is not a general-purpose data-versioning platform. Snapshot and Log-only remain representation baselines. Our results do not establish performance on

public datasets, independent hardware, concurrent workloads, or production systems.

III. REVON DESIGN

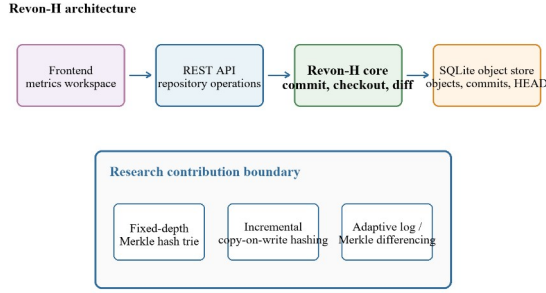


Fig. 1. Revon architecture and research scope.

A. Addressed Object Model

Revon serializes trie nodes and changesets as canonical JSON and names each object by its SHA-256 digest. A commit digest covers the root, parent, changeset, message, and timestamp. Labels such as v1 and v2 are readable aliases only; they do not affect object identity. A durable HEAD reference identifies the current commit.

B. Fixed-Depth Merkle Hash Trie

The main experiment uses $b=8$ and $d=4$. At each level, Revon reads three bits from SHA-256(key), so twelve routing bits select among 4,096 possible leaf buckets. Each bucket retains the full key, allowing Revon to distinguish keys with the same routing prefix. With uniform hashes, expected bucket occupancy is N/b^d . We evaluate other b, d pairs separately rather than treating this default as optimal.

C. Incremental Hashing and Commit

Revon canonicalizes each mutation batch, routes the keys, and groups updates that share trie prefixes. It rewrites each affected leaf once, then rebuilds only the ancestor paths that contain changes. Unchanged child hashes are reused in the

new internal nodes, which preserves structural sharing. A single SQLite transaction writes the new objects, changeset, commit metadata, version alias, measurements, and updated HEAD. On reopen, Revon verifies the stored payload hashes and references.

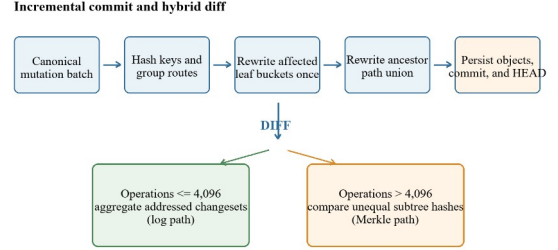


Fig. 2. Commit processing and adaptive diff selection.

D. Hybrid Diff

The log path walks the commit ancestry and aggregates key operations. The Merkle path compares roots recursively, skips subtrees with matching hashes, and examines full leaf entries only under changed paths. Revon-H counts the operations in the requested interval and uses the log path at or below 4,096 operations; above that threshold, it uses Merkle comparison. Revon-M always uses the Merkle path and serves as the ablation.

IV. EXPERIMENTAL METHODOLOGY

A. Variants and Workloads

We compare five systems on the same logical states and mutation batches: Snapshot stores a complete copy per version; Log-only stores an initial state and operation log; Revon-M forces Merkle diff; Revon-H selects its diff path adaptively; and Dolt 2.3.1 uses a keyed SQL table and native JSON diff. We measure Dolt's initial import in two ways: batched SQL INSERT and CSV input through dolt table import -r. The CSV workflow includes serialization in its import time. Both import variants use the same Dolt engine for later operations, and we report their import results separately.

TABLE II. Workloads used in the final evaluation.

Scenario	Rows	Commits	Changed keys	Update locality
Small sparse	1,000	10	10	spread
Medium sparse	10,000	10	10	spread
Medium dense	10,000	10	1,000	spread
Large sparse	100,000	10	100	spread
Application-key local	100,000	10	100	lexically adjacent keys
Range local	100,000	10	100	contiguous sorted-key intervals
Repeated key	100,000	10	100	repeated small key set
Hash-route local	100,000	10	100	shared 9-bit SHA-256 prefix

B. Measurement Contract

We time initial import through the first durable state. For each trial, incremental-commit time is the median across ten commits. Diff and checkout timers include construction of a common output: diff returns sorted changed keys with old and new SHA-256 value hashes as canonical JSON, while checkout returns the full sorted key/value state as canonical UTF-8 JSON. Dolt’s diff time includes JSON parsing and normalization. We check both outputs against the workload oracle. Dolt’s SQL-path commit time includes SQL mutations, add, and commit; we retain commit hashes for historical checkout and create no version tags. Timed operations run without tracemalloc. During each isolated primary trial, an external parent process samples process-tree RSS every 10 ms, including Dolt child processes; sampling happens outside the timed operations. A separate Windows telemetry run records RSS, CPU, read/write bytes, and serial commit throughput in 405 rows (315 measured). Every row passed correctness checks and the schema-2 audit. These counters include setup and correctness work, not just timed operations. The WSL2 replication has incomplete I/O coverage and ran on the same physical host. Work-examined units differ by system and are not normalized.

C. Repetition, Calibration, and Integrity

The primary matrix uses two warm-ups and seven measured trials per configuration, for 120 warm-ups and 420 measured runs across calibration and evaluation. Of these, 280 measured runs belong to the five-system evaluation, including 56 SQL-path Dolt runs; another 56 measured runs use Dolt bulk import. We counterbalance and record model order, and run each system/trial in a clean worker. Each scenario uses one deterministic workload, which the internal audit regenerates and checks by digest. The primary run used Windows 11 build 10.0.26200-SP0 and Python 3.14.3 (run b7a4ec7b610742d9b827f43fd5092fe2). For each scenario and comparison, we divide latency measurements from matching trial numbers and bootstrap the seven paired ratios 20,000 times to obtain a percentile 95% interval. These intervals describe variation across runs on this fixed workload and host; they do not measure variation across datasets or machines. We label intervals that include parity as inconclusive. The 5%

practical margin is an analyst-selected label, not an external standard.

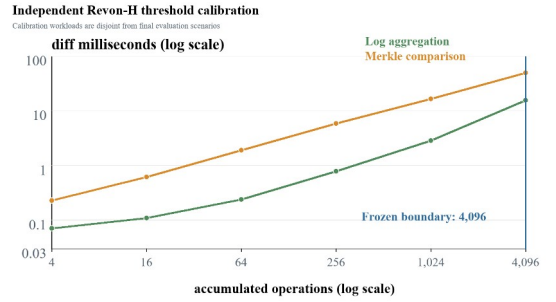


Fig. 3. Threshold calibration; 4,096 is the largest sampled point favoring the log path.

We fixed the 4,096-operation threshold for this machine; it is not a universal crossover. All 540 primary executions completed and passed the state and diff checks. The audit retained all 127 Tukey outlier candidates rather than removing them. A separate sensitivity run varies the seed, payload size, history length, and update pattern; its results are reported apart from the primary tables.

V. RESULTS

A. Final Diff Performance

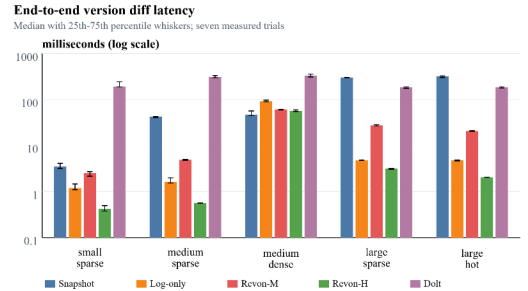


Fig. 4. Diff latency across eight update scenarios; lower values are better.

TABLE III. Revon-H and SQL-path Dolt diff latency; the final column gives the median paired ratio and 95% bootstrap interval.

Scenario	Diff path	Revon-H (ms)	Dolt (ms)	SQL	Dolt/Revon-H ratio (95% CI)
Small sparse	log	0.659	411.69		625.1 (537.1-658.7)
Medium sparse	log	0.814	503.31		482.5 (393.4-652.0)
Medium dense	merkle	50.391	488.42		10.3 (9.0-10.8)
Large sparse	log	46.834	483.08		10.8 (8.2-12.4)
Application-key local	log	39.361	289.87		7.1 (6.8-9.8)
Range local	log	41.359	325.04		7.8 (6.8-11.2)
Repeated key	log	1.992	293.72		141.6 (109.5-173.2)
Hash-route local	log	2.620	278.22		102.4 (97.5-104.0)

In all eight scenarios, the paired Dolt/Revon-H diff ratio exceeded one and its 95% interval excluded parity. Revon-H therefore had lower diff latency in the workflows we tested. Table III gives the median of the seven same-trial ratios and the bootstrap interval. The comparison is between Revon’s in-process API and Dolt’s CLI workflow, which includes Dolt CLI and SQL costs; these ratios are not engine-only speedups. The lexically adjacent application-key workload is distinct from the range-local, repeated-key, and hash-route-local workloads.

B. Hybrid Ablation

Three of the eight primary Revon-M/Revon-H diff intervals included parity, so those comparisons were inconclusive. The separate robustness matrix used 10,000 rows, three seeds, 32- and 128-byte values, histories of 10 and 50 commits, and spread, range-local, and repeated-key updates. A Revon-M/Revon-H ratio above one means Revon-M was slower. On Windows, the ratios for the baseline, 128-byte values, range-local updates, and repeated-key updates were 1.58 [1.23, 1.87], 1.62 [1.38, 1.72], 1.64 [1.38, 1.77], and 2.24 [2.02, 2.38]. The 50-commit interval crossed parity. WSL2 showed the same direction for the other four factors, while the 50-commit interval again overlapped parity. This three-seed sensitivity study on one physical host offers more direct evidence about adaptive selection than the cross-system ratios, but it does not establish general superiority.

C. Commit, Checkout, and Storage

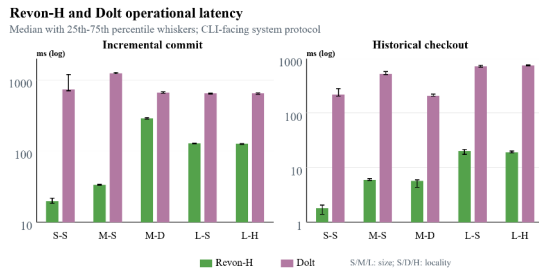


Fig. 5. Operational latency for Revon-H and Dolt; lower values are better.

Across all eight scenarios, the paired Dolt/Revon-H intervals excluded parity for initial import, incremental commit, diff, and historical checkout. Full ratios and intervals appear in paired_ratio_uncertainty.csv. Median initial-import times

across scenario medians were 1,814.83 ms for batched SQL INSERT, 1,701.25 ms for Dolt bulk import, and 1,131.24 ms for Revon-H. In the paired SQL-versus-bulk-import analysis, four scenario intervals favored bulk import and four included parity; the results do not show a general bulk-import advantage. These are workflow measurements, not evidence of engine-level or production-system superiority.

Repository directory size measures the footprint of the full workflow, not storage normalized to the algorithms alone. Across the eight scenarios, Revon-H’s repository footprint was 1.99–6.90 times Dolt’s. After dividing each directory size by the final UTF-8 key/value payload, the median Revon-H/Dolt ratio was 3.82; the corresponding ratio for the residual was 6.82. That residual is not pure metadata: it also includes serialization, indexes, other metadata, and compression effects. After compaction, mean sampled footprints were 22.08 MiB for Revon-M, 22.08 MiB for Revon-H, 2.17 MiB for Dolt SQL, and 2.19 MiB for Dolt bulk import. We compacted Dolt offline with ‘dolt gc’ and the Revon SQLite repositories with ‘VACUUM’; compaction time is excluded from latency and RSS. These are sampled outcomes, not a pure metadata comparison or a universal steady-state result.

D. Trie-Parameter Sensitivity

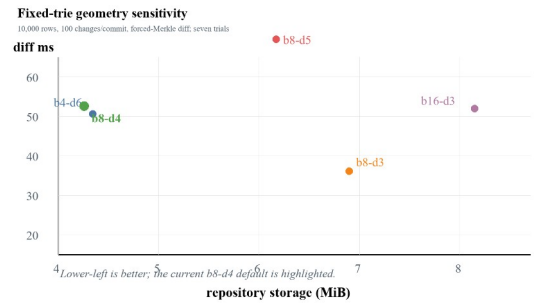


Fig. 6. Fixed-trie sensitivity; points show median diff latency, and the axis extends beyond the largest measured 75th percentile.

The corrected fixed-trie sensitivity run completed 45 executions: 10 warm-ups and 35 measured runs. The b16-d3 configuration had the lowest median diff time (21.46 ms), but used 8.12 MiB. The b8-d4 configuration took 26.74 ms and used the least storage (4.26 MiB). The over-deep b8-d5 configuration took 33.68 ms to diff and used 44.5% more storage than

b8-d4. Thus, b8-d4 balances latency and storage in this sample; it is not a universal optimum.

VI. DISCUSSION

For RQ1, the results show workload-dependent latency and storage trade-offs across the synthetic update patterns we tested. In a separate three-seed study, Revon-H had lower diff times than the forced-Merkle ablation for four factors on both Windows and WSL2. The 50-commit comparison remained inconclusive on both. Lexically adjacent application keys, sorted-range updates, repeated-key updates, and hash-route locality represent different patterns. We did not test ordered range queries, concurrent readers or writers, or production throughput.

For RQ2, the paired Revon-M/Revon-H ablation provides the clearest comparison: both variants use the same durable trie, and differ in diff-path selection. Revon-H showed advantages for the tested short and sparse histories. The 50-commit case, which crossed the fixed operation threshold, showed no resolved difference. These results do not establish a universal advantage for the selector.

For RQ3, the results rule out one trie geometry as best for every workload. Shallow routing reduces internal traversal but leaves larger buckets and higher storage use. Deeper routing reduces bucket occupancy but adds internal-node work. The suitable configuration depends on whether a workload prioritizes latency, storage, or limiting leaf scans.

The Dolt ratios reflect the complete systems and interfaces we measured. Dolt handles SQL parsing, schema management, version-control operations, and production durability, which the Revon prototype does not. Its Prolly Trees also support ordered queries, branches, merges, and remote workflows. Because our timings include each system’s declared interface, they do not isolate the tree algorithms. We therefore cannot conclude that a fixed trie is inherently faster than a Prolly Tree.

VII. LIMITATIONS AND THREATS TO VALIDITY

- Runtime and interface: Revon runs in-process in Python; Dolt is a mature Go executable invoked through CLI and SQL interfaces. This difference affects the measured workflow.
- Feature set: Revon supports one writer and a linear history. It does not support SQL, branches, merges, remotes, schema evolution, or production concurrency.
- Dataset scope: The primary study reaches 100,000 rows. The sensitivity study uses flat key/value tables with 10,000 rows, three seeds, 32- and 128-byte values, histories of 10 and 50 commits, and three update patterns. We did not test alternate schemas, public multi-table datasets, or one-million-row datasets.
- Environment and replication: The primary run used one Windows 11 host. The WSL2 Ubuntu sensitivity run used the same physical computer, not an independent Linux machine. No external researcher or clean clone has reproduced the timings. The internal audit checks evidence consistency; it does not reproduce wall-clock performance.

- Calibration: The log path was faster at the largest sampled operation count that favored it, 4,096. This result does not establish a universal crossover.
- Memory: We measured median whole-worker process-tree RSS in eight primary scenarios for Snapshot, Log-only, Revon-M, Revon-H, and SQL-path Dolt. These measurements include shared workload/oracle state and adapter processes, so they are not product-only memory figures. The parent process sampled every 10 ms outside timed operations. It may miss shorter peaks, and monitoring may add small scheduling pressure. Dolt bulk-import measurements are reported separately from SQL-path Dolt.
- Work units: Keys, log operations, trie-node pairs, and Dolt’s internal work are different quantities. We do not combine them into a normalized work measure.
- Uncertainty: We retained every Tukey outlier candidate. The primary seven-trial ratio intervals are paired bootstrap intervals for each fixed scenario. The additional seed-cluster bootstrap uses only three seeds. Neither analysis estimates performance across a population of datasets or machines.

VIII. FUTURE WORK

A future version could select its structure from observed workload statistics. Key count, prefix skew, mutation density, and leaf occupancy may help choose among the tested trie geometries instead of fixing b and d for every repository. The diff selector could also recalibrate over time, using history distance, changed-prefix density, and observed I/O alongside operation count.

Revon’s storage could be reduced through binary node records, separate value chunks, packed objects, compression, reachability checks, and garbage collection. An ordered Merkle index or content-defined chunking could add range scans and history-independent boundaries while retaining the fixed trie as a controlled baseline.

Moving beyond a linear prototype will require named branches, merge commits, conflict reporting, schema-aware records, concurrent readers and writers, ordered range scans, and remote synchronization. Evaluation also needs public structured datasets, workloads with one million rows, and replications by independent researchers on separate hardware. WSL2 I/O counters remain incomplete, and the Windows sensitivity counters cover the workflow rather than isolating individual operation costs.

IX. CONCLUSION

Revon demonstrates one way to version structured key/value data using immutable objects, incremental Merkle updates, SQLite persistence, historical checkout, and adaptive differencing. Paired intervals and a three-seed sensitivity study show workload-specific differences between Revon-H and the forced-Merkle ablation; the long-history result remains inconclusive. The experiments do not establish performance on real schemas or public datasets, or across independent hardware,

concurrent workloads, and ordered range queries. The evidence describes a prototype, not a production-system replacement.

ACKNOWLEDGMENT

We thank the open-source Dolt project and the researchers whose work on dataset versioning informed our experimental design.

REFERENCES

- [1] S. Chacon and B. Straub, “Git Internals: Git Objects,” in *Pro Git*, 2nd ed., 2014. [Online]. Available: <https://git-scm.com/book/en/v2/Git-Internals-Git-Objects>.
- [2] DoltHub, “Dolt Storage Engine: Block Store,” Dolt Documentation, 2024. [Online]. Available: <https://www.dolthub.com/docs/architecture/storage-engine/block-store/>.
- [3] T. Sehn, “Dolt’s Storage Engine,” DoltHub Blog, Feb. 29, 2024. [Online]. Available: <https://www.dolthub.com/blog/2024-02-29-storage-engine/>.
- [4] R. C. Merkle, “A digital signature based on a conventional encryption function,” in *Advances in Cryptology: CRYPTO ’87, Lecture Notes in Computer Science*, vol. 293, 1988, pp. 369–378, doi: 10.1007/3-540-48184-2_32.
- [5] J. R. Driscoll, N. Sarnak, D. D. Sleator, and R. E. Tarjan, “Making data structures persistent,” *J. Comput. Syst. Sci.*, vol. 38, no. 1, pp. 86–124, 1989, doi: 10.1016/0022-0000(89)90034-2.
- [6] J. Benet, “IPFS: Content Addressed, Versioned, P2P File System,” arXiv:1407.3561, 2014.
- [7] A. Aurolat and F. Taiani, “Merkle Search Trees: Efficient State-Based CRDTs in Open Networks,” in *Proc. 38th IEEE Int. Symp. Reliable Distributed Syst. (SRDS)*, 2019, pp. 221–230, doi: 10.1109/SRDS47363.2019.00032.
- [8] A. Muthitacharoen, B. Chen, and D. Mazières, “A low-bandwidth network file system,” in *Proc. 18th ACM Symp. Operating Syst. Principles (SOSP)*, 2001, pp. 174–187, doi: 10.1145/502034.502052.
- [9] A. Bhardwaj et al., “DataHub: Collaborative Data Science and Dataset Version Management at Scale,” in *Proc. 7th Biennial Conf. Innovative Data Syst. Research (CIDR)*, 2015.
- [10] M. Maddox et al., “Decibel: The Relational Dataset Branching System,” *Proc. VLDB Endowment*, vol. 9, no. 9, pp. 624–635, 2016, doi: 10.14778/2947618.2947619.
- [11] S. Huang, L. Xu, J. Liu, A. J. Elmore, and A. G. Parameswaran, “OrpheusDB: Bolt-on Versioning for Relational Databases,” *Proc. VLDB Endowment*, vol. 10, no. 10, pp. 1130–1141, 2017, doi: 10.14778/3115404.3115417.
- [12] S. Bhattacharjee, A. Chavan, S. Huang, A. Deshpande, and A. Parameswaran, “Principles of Dataset Versioning: Exploring the Recreation/Storage Tradeoff,” *Proc. VLDB Endowment*, vol. 8, no. 12, pp. 1346–1357, 2015, doi: 10.14778/2824032.2824035.
- [13] M. Armbrust et al., “Delta Lake: High-Performance ACID Table Storage over Cloud Object Stores,” *Proc. VLDB Endowment*, vol. 13, no. 12, pp. 3411–3424, 2020, doi: 10.14778/3415478.3415560.
- [14] Attic Labs, “Noms Technical Overview.” GitHub. [Online]. Available: <https://github.com/attic-labs/noms/blob/master/doc/intro.md>.
- [15] S. Wang et al., “ForkBase: An Efficient Storage Engine for Blockchain and Forkable Applications,” *Proc. VLDB Endowment*, vol. 11, no. 10, pp. 1137–1150, 2018, doi: 10.14778/3231751.3231762.
- [16] A. Twigg, A. Byde, G. Milos, T. Moreton, J. Wilkes, and T. Wilkie, “Stratified B-trees and versioning dictionaries,” in *Proc. 3rd USENIX Workshop on Hot Topics in Storage and File Systems (HotStorage)*, 2011. [Online]. Available: <https://arxiv.org/abs/1103.4282>.
- [17] A. Seering, P. Cudré-Mauroux, S. Madden, and M. Stonebraker, “Efficient versioning for scientific array databases,” in *Proc. IEEE 28th Int. Conf. Data Engineering (ICDE)*, 2012, pp. 1013–1024, doi: 10.1109/ICDE.2012.102. [Online]. Available: https://www.researchgate.net/publication/241633210_Efficient_Versioning_for_Scientific_Array_Databases.
- [18] E. Soroush and M. Balazinska, “Time travel in a scientific array database,” in *Proc. IEEE 29th Int. Conf. Data Engineering (ICDE)*, 2013, pp. 98–109, doi: 10.1109/ICDE.2013.6544817. [Online]. Available: https://scidb.cs.washington.edu/paper/ICDE13_conf_full_422.pdf.
- [19] G. S. Brodal, S. Sioutas, K. Tsakalidis, and K. Tsichlas, “Fully persistent B-trees,” *Theoretical Computer Science*, vol. 841, pp. 10–26, 2020, doi: 10.1016/j.tcs.2020.06.027. [Online]. Available: <https://cs.au.dk/~gerth/papers/tcs20.pdf>.
- [20] N. Saquib, C. Krintz, and R. Wolski, “Replicated versioned data structures for wide-area distributed systems,” *IEEE Transactions on Parallel and Distributed Systems*, vol. 34, no. 1, pp. 207–224, 2023, doi: 10.1109/TPDS.2022.3217969. [Online]. Available: <https://racelab.cs.ucsb.edu/papers/tpds22-versioned.pdf>.
- [21] M. E. Schüle, L. Karnowski, J. Schmeißer, B. Kleiner, A. Kemper, and T. Neumann, “Versioning in main-memory database systems: From MusaeusDB to TardisDB,” in *Proc. 31st Int. Conf. Scientific and Statistical Database Management (SSDBM)*, 2019, pp. 169–180, doi: 10.1145/3335783.3335792. [Online]. Available: <https://www-db.cs.tum.edu/~schuele/data/tardis.pdf>.
- [22] P. Buneman, S. Khanna, and W.-C. Tan, “Why and where: A characterization of data provenance,” in *Database Theory: ICDT 2001, Lecture Notes in Computer Science*, vol. 1973, 2001, pp. 316–330, doi: 10.1007/3-540-44503-X_20. [Online]. Available: https://www.research.ed.ac.uk/files/16509989/Why_and_where_A_Characterization_of_Data_Provenance.pdf.
- [23] F. McSherry, D. G. Murray, R. Isaacs, and M. Isard, “Differential dataflow,” in *Proc. 6th Biennial Conf. Innovative Data Systems Research (CIDR)*, 2013. [Online]. Available: https://www.cidrdb.org/cidr2013/Papers/CIDR13_Paper111.pdf.
- [24] M. Akhlaghi et al., “Toward long-term and archivable reproducibility,” *Computing in Science & Engineering*, vol. 23, no. 3, pp. 82–91, 2021, doi: 10.1109/MCSE.2021.3072860. [Online]. Available: <https://arxiv.org/abs/2006.03018>.
- [25] R. Shraga and R. J. Miller, “Explaining dataset changes for semantic data versioning with Explain-Da-V,” arXiv:2301.13095, 2023, technical report/preprint. [Online]. Available: <https://arxiv.org/abs/2301.13095>.
- [26] A. Guo, J. Li, P. Sukprasert, S. Khuller, A. Deshpande, and K. Mukherjee, “To store or not to store: A graph theoretical approach for dataset versioning,” in *Proc. IEEE Int. Parallel and Distributed Processing Symposium (IPDPS)*, 2024, doi: 10.1109/IPDPS57955.2024.00049. [Online]. Available: <https://arxiv.org/abs/2402.11741>.
- [27] A. González-Cebrián, M. Bradford, A. E. Chis, and H. González-Vélez, “Standardised versioning of datasets: A FAIR-compliant proposal,” *Scientific Data*, vol. 11, art. 358, 2024, doi: 10.1038/s41597-024-03153-y. [Online]. Available: <https://www.nature.com/articles/s41597-024-03153-y>.

- [28] J. Zhao and S. Krishnan, “Compression and in-situ query processing for fine-grained array lineage,” arXiv:2405.17701, 2024. [Online]. Available: <https://arxiv.org/abs/2405.17701>.
- [29] H. Gou, Z. Zhang, Y. Sun, P. Xu, F. Tian, L. Wang, and J. Wang, “Git4Data: Database-native version control for AI agents,” arXiv:2609.02106, 2026, preprint. [Online]. Available: <https://arxiv.org/abs/2609.02106>.
- [30] J. Edwards and T. Petricek, “Baseline: Operation-based evolution and versioning of data,” arXiv:2512.09762, 2025, preprint, updated 2026. [Online]. Available: <https://arxiv.org/abs/2512.09762>.

APPENDIX A. REPRODUCIBILITY AND DATA AVAILABILITY

The repository includes the benchmark harness, evidence audit, paired-ratio analysis, multi-seed robustness study, and

manuscript validation tools. The corrected primary paper-profile bundle is `evidence/paper-corrected-issues13-17-counterbalanced-final-20260923`; its paired ratio intervals are in `paired_ratio_uncertainty.csv`. The Windows and WSL2 sensitivity bundles are `evidence/paper-issues18-20-robustness-20260923` and `evidence/paper-issues18-20-linux-wsl-20260923`. Supplemental Windows telemetry is in `evidence/paper-issues14-windows-telemetry-20260924`, and trie sensitivity results are in `evidence/trie-sensitivity-issues8-20260923`. These bundles retain raw rows, manifests, audits, summaries, and seed-cluster intervals. They support checks of evidence consistency; they do not independently reproduce timing results. The project repository is <https://github.com/atharvasheersh/Revon>.